STUDY & EXAM PREPARATION GUIDE

OCP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 21

Streams

CHAPTER OVERVIEW

This chapter covers the following exam objectives: Use Java object and primitive Streams, including lambda expressions implementing functional interfaces, to supply, filter, map, consume, and sort data, Perform decomposition, concatenation and redu...

EXAM OBJECTIVES COVERED

- Use Java object and primitive Streams, including lambda expressions implementing functional interfaces, to supply, filter, map, consume, and sort data
- Perform decomposition, concatenation and reduction, and grouping and partitioning on parallel streams
- Process Java collections concurrently including the use of parallel streams

Chapter 21 Streams

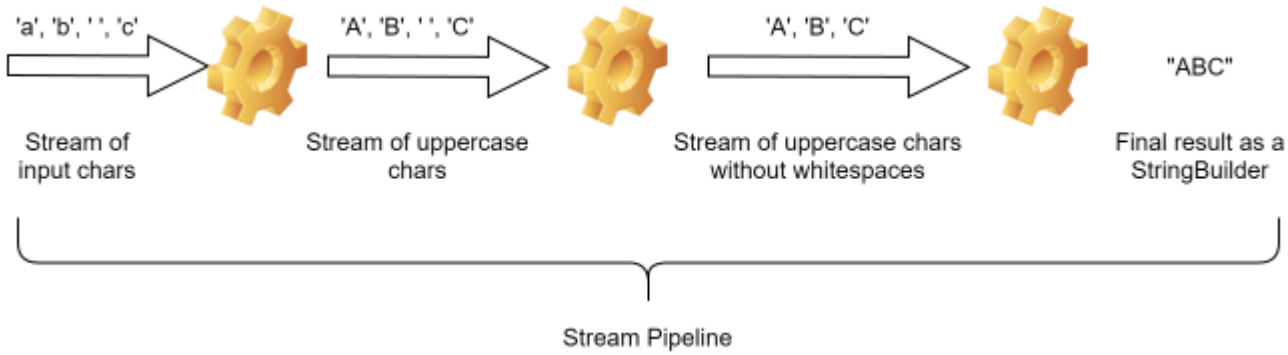
Exam Objectives

- Use Java object and primitive Streams, including lambda expressions implementing functional interfaces, to supply, filter, map, consume, and sort data
- Perform decomposition, concatenation and reduction, and grouping and partitioning on parallel streams
- Process Java collections concurrently including the use of parallel streams

21.1 Stream Basics ☝

In a previous chapter, you learned how to deal with a group of related objects using the Collections API. The Stream API is similar in the sense that it too allows you to deal with a group of related objects but it does so in an entirely different manner. Instead of letting you manipulate the collection itself (using methods such as add and remove), a **stream** provides a **sequential** flow of objects that you can tap into and manipulate on one side, thereby creating a new altered stream or a result on the other side. A **stream pipeline** is a series of connected streams that can be thought of as an assembly line of a manufacturing plant where raw parts come in from one end as an incoming stream, go through a series of transformations one by one, and go out as finished products from the other end.

The developer just needs to just specify the operations in the form on "functions" that must be performed on the objects and the stream pipeline takes care of the actual invocation of those operations on the objects. For example, imagine a stream pipeline where incoming stream of characters is first converted into a stream of upper case characters, which is then filtered to produce a stream of non-whitespace characters and, finally, the characters are joined into a string. Something like this:



The stream pipeline shown in the above image can be coded as follows:

```
List<Character> al = List.of('a', 'b' , ' ', 'c');
Stream<Character> allCharsStrm = al.stream();
Stream<Character> uppercaseStrm = allCharsStrm.map(ch-
>Character.toUpperCase(ch));
Stream<Character> onlyRealCharsStrm = uppercaseStrm.filter(ch-
>!Character.isWhitespace(ch));
StringBuilder finalString = onlyRealCharsStrm.collect( StringBuilder::new ,
(s, ch) → s.append(ch), (s1, s2) → s1.append(s2));
System.out.println(finalString);
```

Don't worry about the method names and the arguments passed to them right now. Focus on the following points instead:

1. The initial stream is being created from a `List` of characters. The list is therefore, the **source** of the initial stream.
2. At each step except the last one, the stream undergoes transformation such that the output is still a stream. Operations that transform a stream into a new stream are called **intermediate operations**.
3. The last step does not produce a stream but a `StringBuilder` object. This is the final result of the stream pipeline. The last operation is called the **terminal operation**.
4. Note that in the above image, all the elements of the stream seem to flow

4. through each junction at once but it is not so in reality. The elements flow through the stream pipeline sequentially, one by one. So, for example, `'a'` will pass through all of the three junctions first and then the next element `'b'` will enter the pipeline, and so on. A stream pipeline will not have more than one element traversing through it at any given moment (unless it is a parallel stream or the operation is stateful, but let's worry about that later). You may verify this by adding print statements in the lambda expressions passed to the calls to the `map`, `filter`, and `collect` methods.

Observe that the return type of intermediate operations is always a `Stream`, so, typically, the code for a stream pipeline is written in a single chained statement like this:

```
StringBuilder finalString = al.stream().map(ch->Character.toUpperCase(ch)).filter(ch -> !Character.isWhitespace(ch)).collect(StringBuilder::new , (s, ch) -> s.append(ch), (s1, s2) -> s1.append(s2));
```

which is often formatted for better readability as:

```
StringBuilder finalString = al.stream()
  .map(ch->Character.toUpperCase(ch))
  .filter(ch->!Character.isWhitespace(ch))
  .collect( StringBuilder::new , (s, ch)->s.append(ch), (s1, s2) ->
s1.append(s2));
```

As you can see, a stream pipeline is nothing more than a chain of method calls but the fundamental difference between this code and other regular methods calls is that here the arguments are not data variables data but functions. Since the functions that we pass to the stream operations as arguments provide behavior to these operations, these parameters are called "**behavioral parameters**".

Another point to understand in the above example is that we have not written any code to iterate through the elements of the list and to pass each element to different functions. The `Stream` implementation class (provided by the JDK) makes the

elements flow through those functions at run time. This is very different from, for example, a `for` loop where we not only write the business logic that is to be applied to each element but also write the code that controls the flow of elements, i.e., the iterations. Thus, with streams, we only need to specify "what" we want to do with each element. The stream framework takes care of "how" that logic is applied to all the elements. In other words, stream operations allows us to "declare" our business logic that we want to apply to all the elements and that is why these stream operations are called "**aggregate operations**".

To sum up, the Stream API allows us to express your logic in a declarative and functional style instead of the imperative style, which, as you will learn later, also makes it easier to parallelize the execution of the code.

Streams for the OCP Exam

The Stream API defines a large number of classes and methods. Moreover, due to the popularity of declarative style of programming, several existing classes from other APIs such the Collections API have also been updated to include methods that deal with streams. With every Java version, new methods are added to the API to cater to a wide range of usecases. It is practically impossible to memorize them all. Fortunately, that is not what the exam expects from you either. It does not trick you by using non-existent methods or by passing non-existent arguments in the code, for example. Instead, it expects you to know what a class or a method used in the code actually does assuming that the class or the method is used correctly. While that might seem too hard at first, it is not. Since the whole point of declarative style of programming is to convey what you actually want done at a higher level (instead of providing detailed steps for doing it), it is usually easy to guess what a method might be doing just by observing its name and the intention of the given code snippet. For example, in the above code, one can easily guess what the `map` and the `filter` methods do. Agreed, it is hard to guess what the `collect` method does but I will cover everything like that by the end of this chapter. For now, just focus on the way of doing things in streams instead of learning and memorizing all the class and method names.

21.1.1 Creating Streams ☝

It is important to understand that a stream does not "store" or "keep" elements within itself. It merely gets them one by one from an underlying source. The source could be anything that can produce elements on demand such as a hardcoded list of objects, a collection, or an array. It could even be a generator function that creates objects itself or fetches them from somewhere else such as a database. Thus, unlike a collection or an array, a Stream does not have the concept of "size" or "length". A stream will continue to flow until there are elements to process. For the same reason, streams are not modifiable either. Since there are no elements "in" a Stream, you cannot add or remove elements to/from a stream. You can modify the stream source if it permits modification but whether the modification will be reflected in the stream or not depends on how the source is implemented.

Streams are represented by the `java.util.stream.Stream` interface. There is no publicly accessible class in the JDK that implements `Stream` and so, you cannot directly instantiate a `Stream` object. However, Java provides several ways to create a `Stream` from various sources. Let me show you a few of the common ones.

The easiest way to create a stream over a known number of objects is to use the static `of()` method. For example:

```
Stream<Integer> s = Stream.of(1, 2, 3, null);//yes, a stream may have
null elements
Stream<String> s = Stream.of("A", "B", "C");
```

Of course, such streams are good for testing purposes only because usually the objects are not hardcoded in the code. In practice, the `stream()` method of the `Collection` interface is used quite often to process a collection as a stream:

```
Collection<String> collection = //get the collection somehow
Stream<String> s = collection.stream();
```

Similarly, the `java.lang.Arrays` class has several overloaded `stream()` methods that return a `Stream` view of the given array. There is even a `Stream.empty()` method that creates a `Stream` over zero elements.

Creating a stream over an unknown number of objects ☝

`Stream` has a static `generate(Supplier<? extends T> s)` method that sources objects for the stream from the given generator function. Of course, if the generator function is such that it keeps generating objects forever, the resulting stream would be an **infinite stream**. For example, `Stream<Double> sd = Stream.generate(Math::random);` would create a never ending stream of `Double`s.

Another way to source objects for the stream dynamically is to use one of the `Stream`'s `iterate` methods:

```
Student first = fetchNextStudentFromDB(0);
Stream<Student> ss = Stream.iterate(first,
  previous → fetchNextStudentFromDB(previous.id())
  );
```

The above code creates a stream of `Student` objects by fetching them from a database using a hypothetical `fetchNextStudentFromDB` method. The interesting part here is that the method uses the information from the previous element in the stream to fetch the next element.

Creating a stream by concatenating other streams ☝

`Stream` has a static `concat(Stream s1, Stream s2)` method that concatenates elements of two streams into a new stream. For example:

```
Stream<Integer> s1 = Stream.of(1, 2);
Stream<Integer> s2 = Stream.of(2, 3);
Stream<Integer> s  =  Stream.concat(s1, s2);//1, 2, 2, 3
```

The concatenated stream has elements of the first stream followed by the second. The concatenated stream is ordered if both of the input streams are ordered, and parallel if either of the input streams is parallel.

21.1.2 Stream operations ☝

The `Stream` interface contains several methods to process the elements flowing through the stream. These are called **stream operations**. Stream operations are of two types - **intermediate operations** and **terminal operations**. Intermediate operations return a new, potentially altered, stream, which allows the next operation in the pipeline to be chained, while the terminal operations produce the final result and denote the termination of the stream pipeline. This also means that you can't invoke any operation on a stream after invoking a terminal operation. Knowing whether a stream operation is intermediate or terminal is very important because intermediate operations are always lazy and do not actually do anything until they are forced to execute by a terminal operation chained at the end of the pipeline.

Intermediate operations are further classified into **stateless** and **stateful** operations. Stateless operations, such as `filter` and `map` process each element independently without considering any information derived from the elements processed earlier. Stateful operations, such as `distinct` and `sorted`, typically keep a state that incorporates information from each element as it is processed and that state, in turn, affects the processing of subsequent elements.

Although the OCP exam does not focus on performance you should be aware that stateful operations have serious performance implications on a stream pipeline. This is because stateful operations may have to process the entire input before producing even the first element of the output stream (as is the case with `sorted`), and/or may also require to hold a significant amount of data while processing the elements (as is the case with `distinct`). They can also cause bottlenecks when the execution of a stream pipeline is parallelized.

The following table lists the important intermediate as well as terminal operations that you need to be aware of for the exam.

Intermediate Operations

Stateless:

`filter(Predicate<? super T> predicate)`, `map(Function<? super T,? extends R> mapper)`, `limit(long maxSize)`, `peek(Consumer<? super T> action)`

Stateful:

`sorted()`, `sorted(Comparator<? super T> comparator)`, `distinct()`, `takeWhile(Predicate p)`, and `dropWhile(Predicate p)`

Terminal Operations

Methods that return boolean:

`allMatch(Predicate<? super T> predicate)`, `anyMatch(Predicate<? super T> predicate)`, `noneMatch(Predicate<? super T> predicate)`

Methods that return Optional:

`findAny()`, `findFirst()`, `max(Comparator<? super T> comparator)`, `min(Comparator<? super T> comparator)`

Others:

`long count()`, `void forEach(Consumer<? super T> action)`, `Iterator<T> iterate()`, `toList()`, `toArray()`, and various overloaded `reduce` and `collect` methods.

Let us now see how these methods work, starting with intermediate operations first.

21.1.3 Intermediate Operations ☝

Here is an overview of the important intermediate stream operations. Remember that since intermediate operations are lazy, the examples will not produce any output unless you apply a terminal such as `forEach(System.out::println)` at the end.

Filtering a stream ☝

The `filter` method inspects each element of the incoming stream and discards those elements that do not satisfy the given predicate. Thus, only those elements for which the `Predicate` returns `true` are propagated to the next stage of the pipeline. For example, if `s` points to a `Stream<Character>` object, `s.filter(ch->!Character.isWhitespace(ch))` causes all whitespace characters to be discarded.

Mapping a stream ☝

The `map` method replaces an incoming object with an object returned by the given `Function` in the outgoing stream. For example, if you have a `Stream` of ids of `Student` objects, you can transform it into a stream of `Student` objects like this:

```
Stream<Student> ids = Stream.of(1, 2, 3).map( id→ getStudentById(id) );
```

Besides map, `Stream` has a `flatMap(Function<? super T,? extends Stream<? extends R>> mapper)` method as well which works similarly but instead of directly mapping an incoming element to a new element, it uses the mapping function to map an incoming element to a stream of elements and puts the elements of that stream into the outgoing stream. For example: The following code maps a stream of incoming course ids to a stream of students enrolled in those courses:

```
Function<Integer, Stream<Student>> getStudentsByCourseId = (courseId)→{
  List<Student> l = getEnrolledStudents(courseId); //fetch the list from DB
  return l.stream();
};
```

```
Stream<Student> students = Stream.of(101, 102,
103).flatMap(getStudentsByCourseId);
```

Observe that the `getStudentsByCourseId` is a `Function` that takes a `courseId` and returns a `Stream` of `Student`s enrolled in that course. This function is then passed to the `flatMap` method.

Limiting a stream ☝

The `limit` method truncates a stream to the given number of elements. This method is helpful to limit the number of elements of a large or an infinite stream. For example, the following code limits the resulting stream to only 10 random numbers:

```
Stream<Double> sd = Stream.generate(Math::random).limit(10);
```

Peeking into a stream ☝

The `peek` method allows you to add a `Consumer` in a stream pipeline without altering the incoming stream in any way. This method can be used to log each element as it passes through that point in the stream pipeline. For example,

```
Stream<Double> sd = Stream.generate(Math::random).peek(d→{ logger.debug("Got double "+d);});
```

Sorting a stream ☝

There are two overloaded `sorted` methods. The one without any parameter sorts the incoming objects using their natural order. Recall that for objects to be sorted by their natural order, the objects must implement the `java.lang.Comparable` interface. If the class does not implement `Comparable`, you can use the `sorted` method that takes a `java.util.Comparator` object. For example, the following statement generates a sorted stream of 10 random numbers:

```
Stream<Double> sd = Stream.generate(Math::random).limit(10).sorted();
```

Since a stream cannot be sorted until all of the elements are known, `sorted` is a stateful operation and must be used cautiously on large streams. Applying it on infinite streams will cause the program to run out of memory.

Removing duplicates using `distinct` ☝

The `distinct` method filters out the duplicate elements from the incoming stream. Objects are compared using the `equals` method. This too is a stateful operation and must be used cautiously on large streams. For example, the following statement removes duplicate numbers from the incoming stream of random numbers:

```
Stream<Double> sd = Stream.generate(Math::random).limit(10).distinct();
```

21.1.4 Understanding Optional ☝

Since many of the terminal operations return `Optional`, here is a primer on this class.

Java has been derided since its inception for not providing a good way to handle `null` values. Typically, it is the developer's responsibility to check a reference for `null` before invoking any methods on it, failing which, the code may end up throwing a `NullPointerException` at run time. For example, the call to `s.enrollInCourse("Math")` in the following code will throw a `NullPointerException` if `getStudentById(10)` returns `null`.

```
Student s = getStudentById(10);
s.enrollInCourse("Math");
```

Ideally, the developer should have checked `s` for `null` and should have provided an alternate execution path before invoking `enrollInCourse()` on it. Something like

`if(s≠null) s.enrollInCourse("Math") else print("No such student");`. However, there are at least three issues with this approach. **First**, the compiler does not enforce `null` checks before any reference is used and that allows the developer to get away with writing null-unsafe code. **Second**, writing `null` checks everywhere is not only tedious but also reduces the readability of the code. **Third**, code embedded within an if/else statement does not fit into the functional programming style where multiple calls are chained together.

The `java.util.Optional` class introduced in Java 1.8 is Java's solution to this problem. `Optional` is a wrapper class with several convenience methods that make is easy to deal with null as well as non-null values. The way this solution works is that instead of returning a reference directly, a method returns an `Optional` that wraps the reference that it wants to return. The caller of the method can now be sure that the method will never return `null` and can use the `isPresent` or `isEmpty` methods to check if the `Optional` actually contains a value and then use its `get` method to get that value:

```
Optional<Student> os = getStudentById(id);
if(os.isPresent()){
  Student s = os.get();
  s.enrollInCourse("MATH");
}
```

Note that calling `get()` on an empty optional throws a `java.util.NoSuchElementException`, which is an unchecked exception, so, the call to `get()` should always be made after confirming that the optional is not empty.

Although the above code isn't any better than the one that uses a null check, but `Optional` has several methods that can be used to provide an alternate path of execution if an optional is empty without using an `if` statement. For example, the following code uses the `ifPresentOrElse` method to achieve the same thing as above:

`Optional<Student> os = getStudentById(10);`

`os.ifPresentOrElse(s→s.enrollInCourse("MATH"), ()→{ print("No such student"); });`

The `ifPresentOrElse` method has two parameters - a `Consumer`, which is invoked if the `Optional` does contain a non-null value and a `Runnable` that is executed if the `Optional` is empty.

You should check out these other methods of the `Optional` class from the JavaDoc - `or`, `orElse`, `orElseGet`, and `orElseThrow`.

Creating an Optional ☝

An `Optional` can be created using one of the three static methods of the `Optional` class, namely, `empty()`, `of(T value)`, and `ofNullable(T value)`. For example:

```
Optional<String> os = Optional.empty();
String str = "value";
Optional<String> os = Optional.of(str); //str must not be null
Optional<String> os = Optional.ofNullable(str); //str may be null
```

21.1.5 Terminal operations ☝

The following is an overview of the important terminal operations.

Operations that return `boolean` ☝

Terminal operations that return `boolean` are fairly straightforward - `allMatch` returns `true` if the given `Predicate` returns `true` for all of the elements in the stream, `anyMatch` returns `true` if the given `Predicate` returns `true` for any of the

elements in the stream, and `noneMatch` returns `true` if the given `Predicate` returns `false` for all of the elements in the stream.

You need to be aware of two points about these methods though. The first is that they are all **short-circuiting** operations. Meaning, if the result of the operation can be determined conclusively during the evaluation of any element, the stream pipeline may stop executing the pipeline for the rest of the elements. For example, `Stream.of(2, 4, 6).peek(System.out::print).allMatch(i -> i%2==1);` will skip the execution of the pipeline for elements `4` and `6` because the result can be determined after the evaluation of the pipeline for element `2` itself. Therefore, the `peek` operation will execute only for the element `2` even though it appears before the terminal operation. Their short-circuiting nature can even terminate the processing of infinite streams. For example, `Stream.generate(Math::random).peek(System.out::println).anyMatch(d -> d>0.5);` will terminate as soon as `Math.random()` returns a value greater than `0.5`. But, of course, relying on such logic is dangerous because you never know how long it will take for a stream to encounter a value that satisfies the given `Predicate`.

The second is that `allMatch` and `noneMatch` return `true` for empty streams but `anyMatch` returns `false`.

Operations that return `Optional` ☝

The `findAny` and `findFirst` methods are short-circuiting operations that return any or the first value of the stream respectively wrapped in an `Optional`. While the `findFirst` returns the first element of the stream if the stream is ordered (usually, streams created using an ordered collection such as `List` is ordered), the `findAny` method is free to return any element.

The `max` and `min` methods evaluate every element of the thread before returning the maximum and the minimum values wrapped in an `Optional` respectively.

All four methods return an empty `Optional` if the stream is empty.

Counting elements in a stream ☝

The `count()` method returns the number of elements in the stream as a `long` value. For example, `Stream.of(1, 2, 3).filter(i→i%2==0).count()` returns `1`. Note that it returns `0` for an empty stream.

Since a stream does not know how many elements are going to pass through it, you have to be careful while invoking methods such as `min`, `max`, and `count`, that need the stream to end before they can return their result. Invoking such methods on an infinite stream will cause the program to crash.

Accumulating elements into a List or an array ☝

The `toList` method accumulates the elements of this stream into an **unmodifiable List** and the `toArray` method accumulates the elements into an `Object[]`.

Processing each element individually using `iterator` and `forEach` ☝

The terminal operations that you saw above are basically just readymade functionality that is available to use right out of the box. There is nothing special about those methods except that the Java designers felt that these methods would be useful and made them available in the standard library. In fact, I would rather let a database perform these operations than use the Stream API. Having said that, there are situations where there is no option but to process each data element within the Java application. There are two methods in `Stream` that let you do so - `iterator` and `forEach`. Let's look at `iterator` first.

Although a `Stream` is not an `Iterable` (which means, you can't directly loop over its values using the enhanced for loop), the `iterator()` method allows you to "loop through" the values of a stream in the imperative style of programming by returning an `Iterator`. This is useful when you want to break out of stream processing upon

some condition. For example, the following code breaks the while loop after encountering the element `2`:

```
Iterator<Integer> it = Stream.of(1, 2, 3).iterator();
while(it.hasNext()){
  Integer i = it.next();
  System.out.println("Processed "+i);
  if(i == 2 ) break;
}
```

The `void forEach(Consumer<? super T> c)` method, on the other hand, is useful when you want to apply your business logic to every element in the stream without caring about the order or the number of elements in the stream. For example: `Stream.of(1, 2, 3).forEach(System.out::print);` prints each element. Although it seems no different from an Iterator, you will learn soon that `forEach` is better suited to consume elements of the stream in parallel. But note that there is no way to "break" the processing of elements at a precise location when using `forEach` (except by throwing an exception but even that will not work as expected when the elements are being processed in parallel). The `forEach` and `iterator` methods highlight the difference between aggregate and normal operations (`forEach()` is an aggregate operation while `iterator()` is not).

Stream reduction using reduce ☝

The purpose of reduction is to process the elements of the stream such that a single valued result is obtained. There are three overloaded `reduce` methods for this purpose and all of them work on the same principle, which is to create a new result value every time an element of the stream is processed using an "accumulator" function. The accumulator function takes the current result and the next element to produce the new result value. Let's check out how the three reduce methods work.

1. `Optional<T> reduce(BinaryOperator<T> accumulator)`
This version is used to reduce a stream when the type of the objects in the stream and

the type of the result are the same. For example, this is perfect if you want to compute the sum of the values of a stream of `Integer`s. Here, the accumulator function would be a `BinaryOperator` that takes the current sum and the next stream element and returns the new sum.

```
Optional<Integer> sum = reduce( (oldsum, newelement) -> {
  int newsum = oldsum + newelement;
  System.out.println("Old sum = "+oldsum+" New element = "+newelement+"
New sum = "+newsum);
  return newsum;
});
System.out.println("sum = "+sum);
```

It produces the following output:

```
Old sum = 1 New element = 2 New sum = 3
Old sum = 3 New element = 3 New sum = 6
sum = Optional[6]
```

Observe that the following points in the above output:

1. The accumulator function is invoked only twice and not thrice, even though there are three elements in the stream. This is because at the beginning, the result is the same as the value of the first element. Thus, the accumulator function is needed only from the second element onwards. Note that if the stream had only one element, the value of the first element itself would be the final result.
2. The method returns an `Optional`. As explained earlier, returning an `Optional` avoids returning a `null` when there is no result to return. Here, the `reduce` method returns an empty `Optional` if the stream is empty.

2. `T reduce(T identity, BinaryOperator<T> accumulator)`
This version too is used to reduce a stream when the type of the objects in the stream and the type of the result are the same. But instead of initializing the result with the value of the first element, it initializes the result with the identity value passed as the

first argument. Another difference is that this version does not return an `Optional`. It returns the identity value if the stream is empty.

```
Integer sum = reduce( 0, (oldsum, newelement) -> {
  int newsum = oldsum + newelement;
  System.out.println("Old sum = "+oldsum+" New element = "+newelement+"
New sum = "+newsum);
  return newsum;
});
System.out.println("sum = "+sum);
```

It produces the following output:

```
Old sum = 0 New element = 1 New sum = 1
Old sum = 1 New element = 2 New sum = 3
Old sum = 3 New element = 3 New sum = 6
sum = 6
```

Observe that the following points in the above output:

1. The accumulator function is invoked thrice.
2. The method returns an `Integer` instead of `Optional`.
3. Although not evident from the above output, be aware that it will throw a `NullPointerException` if you pass `null` as the identity argument.

You will see later that the value picked for identity plays an important role when the stream is parallelized.

3. `U reduce(U identity, BiFunction<U,? super T, U> accumulator, BinaryOperator<U> combiner)`

The signature of this version looks intimidating but trust me, it is no monster.

First, observe that the type of the `identity` value (`U`) is different from the type of the objects in the stream (`T`). Thus, this version of reduce allows the type of the result to be different from the type of the objects. The change of type (from the type of the objects in the stream to the type of the result) happens in the accumulator function.

Second, remember that `BinaryOperator` extends `BiFunction` and is therefore, just a special case of `BiFunction`. While a `BiFunction` can be applied to two arguments of different types and can produce a result of yet another type, `BinaryOperator` restricts `BiFunction` to work with just one type. The parameters types and the return type of a `BinaryOperator` are always the same. For example, let's say we want to sum of a stream of `Integer`s to be a `Long` instead of an `Integer`. The accumulator function, in this case, must be able to "add" two values of different types (`Long` and `Integer`) and produce the result of the required type (`Long`). Here is how this accumulator function can be implemented using a lambda:

```
(Long oldsum, Integer newelement) -> {
  Long newsum = Long.sum(oldsum, newelement.longValue());
  return newsum;
}
```

Observe that I have specified the type of the parameters explicitly in the above expression only to highlight the fact that they are different. They are usually omitted in practice.

Now, coming to the third parameter named `combiner`. This parameter is used when a stream is split into multiple fragments so that each fragment can be processed separately in parallel. The result of each fragment is combined using the combiner function to arrive at the final result. Thus, the purpose of the combiner function is to add the results of any two fragments and produce the combined result of the two fragments. Since types of the results of the fragments and the type of the final result are always the same, i.e., `U`, the combiner function is just a `BinaryOperator` instead of a `BiFunction`. You can ignore this argument for now because we do not expect the sample stream to be processed in parallel at this time but here is how the combiner function can be implemented in this case:

```
(Long sumOfFragment1, Long sumOfFragment2) -> {
    return Long.sum(sumOfFragment1, sumOfFragment2);
}
```

Here is the complete code for the summation:

```
Long sum = Stream.of(1, 2, 3).reduce(Long.valueOf(0),
  (oldsum, newelement) -> {
    Long newsum = Long.sum(oldsum, newelement.longValue());
    System.out.println("Old sum = "+oldsum+" New element = "+newelement+"
New sum = "+newsum);
    return newsum;
  },
  (sum1, sum2)-> {
    System.out.println("Combining "+sum1+" "+sum2);
    return Long.sum(sum1, sum2);
  });
System.out.println("sum = "+sum);
```

It produces the following output:

```
Old sum = 0 New element = 1 New sum = 1
Old sum = 1 New element = 2 New sum = 3
Old sum = 3 New element = 3 New sum = 6
sum = 6
```

Observe that there is no difference in the output.

Mutable reduction using collect ☝

A mutable reduction operation is similar to the regular reduction operation discussed above in the sense that it too processes the elements of the stream such that a single valued result is obtained. However, instead of creating and returning a new result value after digesting an element, the mutable reduction updates the existing result object itself. Thus, the same result object is updated repeatedly for each element of the stream. It is then returned after all the elements are processed. Take a look at the signature of the collect method now:

`R collect(Supplier<R> supplier, BiConsumer<R,? super T> accumulator, BiConsumer<R, R> combiner)`

The **supplier** function is used by the stream pipeline to create the mutable result object of type `R`. The mutable result object returned by the supplier function will be updated every time a stream element is processed. The **accumulator** and the **combiner** functions play the same role as they do in the reduce methods. The accumulator function applies a stream element of type `T` to the result object and the combiner function joins the result objects produced by different fragments of a stream if the stream is split while being processed in parallel.

Let's see how this works step by step if we want to join a stream of characters into a `StringBuilder`.

1. Since the final result will be of type `StringBuilder`, we need a `Supplier` function that will return an empty `StringBuilder`. This can be done easily with a constructor reference `Supplier<StringBuilder> supplier = StringBuilder::new;` But let's use a lambda with a body so that we can include a print statement in it:

```
Supplier<StringBuilder> supplier = () -> {
  System.out.println("Creating a new StringBuilder");
  return new StringBuilder();
};
```

2. The accumulator function should be a `BiConsumer` that knows how to take an element of the stream (i.e. a `Character`) and add it into the existing result object (i.e. a `StringBuilder`). This can also be done using the method reference `StringBuilder::append` but let's go with the following instead:

```
BiConsumer<StringBuilder, Character> accumulator = (sb, ch) -> {
```

2.
```
  System.out.println("Appending "+ch+" to "+sb);
  sb.append(ch);
};
```

3. The combiner function should also be a `BiConsumer` but unlike the accumulator, the combiner should know how to update the result of a stream fragment with the result of another stream fragment. Let's implement it like this:

```
BiConsumer<StringBuilder , StringBuilder> combiner = (s1, s2) -> {
  System.out.println("Combining "+s1+" and "+s2);
  s1.append(s2);
};
```

Observe that the combiner function should "add" the contents of the second argument into the first one (i.e. `s2` into `s1`) and not the other way round. The object pointed to by the second argument will eventually be discarded by the underlying stream framework.

Thus, assuming that you have already defined the variables as above, the collect method call would be `collect(supplier, accumulator, combiner)`. Note that in professional code, you will usually call the method without defining the variables explicitly. Like this: `collect(StringBuilder::new, StringBuilder::append, StringBuilder::append);` .

Compare these accumulator and the combiner functions with the ones used for the reduce method. The ones used for `collect` don't return any value (because they are `BiConsumer` s), while the ones used for reduce do (because they are `BinaryOperator` s or `BiFunction` s)

To try it out, you can run the following code snippet from main:

```
Stream<Character> stream = Stream.of('a', 'b', 'c');
StringBuilder result = stream.collect(supplier, accumulator,
```

```
combiner);
System.out.println("result = "+result);
```

It produces the following output:

```
Creating a new StringBuilder
Appending a to
Appending b to a
Appending c to ab
result = abc
```

The output bears out what we discussed earlier. Only the combiner is never invoked because the stream is not being split in this example but don't worry, you will see it used later.

21.1.6 Quiz ☝

Q 1. Given:
`Stream<Integer> marks = Stream.of(55, 65, 70, 80, 70);`
What will be the output of the following lines of code assuming they are executed independently from each other?

1. `System.out.println(marks.sorted());`
2. `System.out.println(marks.map(i -> ""+i).max());`
3. `System.out.println(marks.peek(System.out::println).sorted().peek(System.out::println ).count());`
4. `System.out.println(marks.peek(System.out::println).distinct().findAny());`
5. `System.out.println(marks.peek(System.out::println).sorted().findFirst());`
6. `System.out.println(marks.peek(System.out::println).filter(i -> i<0).count());`

Answer:

1. Remember that `sorted` is an intermediate operation. It does not trigger the execution of the stream pipeline until a terminal operation is applied at the end of the pipeline. Therefore, it will just print something like `java.util.stream.SortedOps$OfRef@3a71f4dd`. To print the elements of the stream, you can do `marks.sorted().forEach(System.out::println);`.
2. The `map` operation translates the incoming stream of `Integer`s to a stream of `String`s.
   Unlike `sorted`, `max` is a terminal operation and so, it will trigger the execution of the stream pipeline. However, also unlike `sorted`, `max` and `min` require a `Comparator` object to be passed as an argument. Thus, this statement will not compile. You can pass `String::compareTo` to `max` and then it will print `Optional[80]`. Remember that `max` and `min` return an `Optional`.
3. If you expected the `peek` operations of the given pipeline to produce any output, you are in for a surprise. The stream execution is smart enough to figure out that none of the intermediate operations in the given pipeline change the count of the elements in the stream and so, it short circuits all of them and returns the count based on the initial stream itself. Thus, it will just print `5`. Observe that `count()` does not return an `Optional`.
4. This has the same issue as the previous statement but with a slight difference. Instead of optimizing away the intermediate operations altogether, here, the stream pipeline execution optimizes away the processing of all but the first element of the stream because to generate the result of `findAny`, it requires just one element to process. Therefore, only one element passes through the pipeline. Thus, `peek` prints `55` and `findAny` results in `Optional[55]`.
   These two statements highlight the unreliability of the `peek` operation. It is good only for debugging purpose and must not be relied upon for any side affect.
5. The `sorted` operation requires all the elements to be processed before it can generate the first element of the outgoing stream. Therefore, all the elements will pass through `peek` first then through `sorted`. The `findFirst` operation will return `Optional[55]` at the end.
6. Here, the number of elements in the last outgoing stream cannot be determined without executing the `filter` operation for each incoming element. Therefore,

6. each element must pass through `peek` and then through `filter` before `count` can generate the result. The given `peek` operation prints each element. Since no element satisfies the predicate given to the `filter` operation, the outgoing stream from `filter` will be empty. Hence, count will return `0`.

Q 2. Given:

```
Stream<Integer> marks = Stream.of(55, 65, 70, 80, 70);
```

What will be the output of the following lines of code assuming they are executed independently from each other?

1. `System.out.println(marks.limit(2).anyMatch(i -> i>70));`
2. `System.out.println(marks.limit(10).noneMatch(i -> i>70));`
3. `System.out.println(marks.allMatch(i -> i>70).count());`
4. `System.out.println(marks.filter(i -> i>90).limit(2).findFirst());`
5. `System.out.println(marks.filter(i -> i>70).limit(2).count());`

Answer:

1. `limit(2)` will limit the stream to the first two elements, i.e., `55`, `65`. Since neither of the elements satisfy the predicate `i>70`, `anyMatch(i -> i>70);` will return `false`.
2. Since the `limit` method limits the incoming streams to **at most** the given number of elements and since the stream in this example contains only five elements, `limit(10)` will allow all of the five elements to be passed to the outgoing stream. However, the fourth element is greater than 70 and so, `noneMatch(i -> i>70);` will fail at that element and will return `false`.
3. Remember that `allMatch()` is a terminal operation and no other operation can further be chained to it. Also, `allMatch()` returns a `boolean`, which is a primitive value, and calling any method on primitive value will cause compilation failure.
4. `filter(i -> i>90)` causes only the elements that are greater than `90` to pass

4. through. Since there are no such elements in the incoming stream, the outgoing stream from the filter method will be empty. Invoking `limit(2)` on this stream will have no effect because the incoming stream ends with zero elements. Therefore, `findFirst()` will return an empty `Optional`, which will be printed as `Optional.empty`.
   Note that the `limit` method ends the outgoing stream only in two conditions - if the limit is reached or if the incoming stream ends before the limit is reached. So, for example, the stream coming out of the `limit` method in `stream.generate(Math::random).filter(d -> d>1.0).limit(1)` will be a never ending stream that has no elements. This is because the stream generated using `Math::random` never produces a value that is greater than `1.0`. Thus, the incoming stream to `filter(d -> d>1.0)` will keep sending values to the `filter` operation but the `filter` operation will never let any value pass through and so, the limit operation will keep on waiting forever.
5. `filter(i -> i>70)` causes only the elements that are greater than `70` to pass through. Since there is only one such element in the incoming stream, the outgoing stream from the `filter` method will have only one element. Invoking `limit(2)` on this stream will let at most two elements to pass through but the incoming stream ends after only one element, and so, the `count()` will return `1`

Q 3. Given:

```
Stream<String> words = Stream.of( "a", "b", "c", "a", "b");
String targetWord = "a";
```

What will be returned (the type as well the value) by the following statements assuming they are executed independently from each other?

1. `words.reduce(0, (c, w) -> targetWord.equals(w)?c+1:c, (c1, c2) -> c1+c2);`
2. `words.map(s -> targetWord.equals(s)?1:0).reduce((a, b) -> a+b)`
3. `words.map(s -> targetWord.equals(s)?1:0).reduce(Integer::sum)`
4. `words.filter(w -> targetWord.equals(w)).count();`

Answer:
All four statements return the number of times `targetWord` appears in the stream. So, all of them return a value of `2`. However, the version of the `reduce` method that takes just one parameter returns an `Optional` while the other two versions directly return the value. Therefore, options 2 and 3 return an `Optional<Integer>` containing the value `2` while option 1 returns `int` value `2`.
The `count()` operation returns a `long`.

Q 4. Given the declarations of `words` and `targetWords` as above, the following code attempts to count the number of times the target word occurs in the incoming stream. What is wrong with it?

```
Integer count = words.collect(
  ()→0,
  (oldCount, word) → { if(targetWord.equals(word)) oldCount++; },
  (c1, c2) → c1 + c2 );
System.out.println("ccc ="+count);
```

Answer:
Remember that collect performs a **mutable** reduction. It means, the object supplied by the `Supplier` function acts as container that stores the result and the same object is supposed to be updated after each element is processed. Here, an `Integer` object is being supplied by the `Supplier` function but `Integer` is immutable. You can do `oldCount++` but that won't update `oldCount`. Furthermore, the `collect` method accepts `BiConsumer`s as the second and the third arguments. A `BiConsumer` just consumes two values, it doesn't return anything. So, the third lambda doesn't implement `BiConsumer` correctly.

The following is how this should be done:

```
AtomicInteger count = words.collect(
```

```
  AtomicInteger::new,
  (oldCount, word) -> { if(targetWord.equals(word)) oldCount.incrementAndGet();
},
  (c1, c2) -> c1.getAndAdd(c2.get()));
```

In the above code, an `AtomicInteger` is being used as the result container. It is updated after each element is processed. The third lambda correctly folds the value of `c2` into `c1`.

21.1.7 Using primitive streams ☝

The regular object based streams are generally sufficient for all practical purposes but Java designers felt that having specialized primitive streams for `int`, `long`, and `double` would allow a few additional ready-to-use methods such as `sum()` amd `average()` that are applicable only to numeric data to be included in the JDK for the ease of Java developers. Primitive streams also offer better performance as compared to a regular `Stream` of primitive wrapper objects in certain situations. So, the `java.util.stream` package includes the following three interfaces: `IntStream`, `LongStream`, and `DoubleStream`.

These interfaces do not extend `Stream` but they define all the methods of `Stream` that we discussed earlier. The difference between the methods of the `Stream` interface and the methods of the primitive stream interfaces is only in the return types and the types of the parameters. The types used in the methods of the primitive streams are primitive specific versions of the types used in the methods of the `Stream` interface. For example, if you compare the `findAny` method of `Stream` and `IntStream`, you will notice that `Stream`'s `findAny` returns `Optional` while `IntStream`'s `findAny` returns `OptionalInt`. Similarly, `Stream`'s `filter` accepts a `Predicate` while `IntStream`'s `filter` accepts an `IntPredicate`. The same goes for the `generate` method. `Stream`'s `generate` accepts a `Supplier`, while `IntStream`'s `generate` accepts an `IntSupplier`.

The primitive specific optional classes such as `OptionalInt` too work the same way as the regular generic `Optional` class but they have a few changes that tie them to the specific primitive type they are meant to represent. For example, instead of `get()`, `OptionalInt` has `getAsInt()`, `OptionalLong` has `getAsLong()`, and `OptionalDouble` has `getAsDouble()`. These methods return `int`, `long`, and `double` respectively. The OCP exam does not trick you on the method names and you can easily guess what a method does by its name.

With primitive streams, some operations can be performed in less amount of code. For example, instead of writing `int sum = Stream.of(2, 3, 5, 9, 6).reduce(0, (a, b)→a+b);`, you can write `int sum = IntStream.of(2, 3, 5, 9, 6).sum();` Computing the average of a regular stream of `Integer`s would require you to write the accumulator and combiner functions yourself, but if you use an `IntStream`, you can just call `average()` on it: `OptionalDouble average = IntStream.of(2, 3, 5, 9, 6).average();`

Another interesting method in primitive stream classes is the `summaryStatistics()` method, which returns an instance of the `IntSummaryStatistics` (or `LongSummaryStatistics` and `DoubleSummaryStatistics`, depending on which primitive stream you are using) class. This method too is just a ready-to-use utility method to get the sum, average, min, and max of the given primitive stream. For example, the following code prints: `count = 5, sum = 25.0, min = 2.0, max = 9.0, avg = 5.0`:

```
DoubleStream ds = DoubleStream.of(2.0, 3.0, 5.0, 9.0, 6.0);
DoubleSummaryStatistics dss = ds.summaryStatistics();
System.out.println("count = "+dss.getCount()+", sum = "+dss.getSum()+", min
= "+dss.getMin()+", max = "+dss.getMax()+", avg = "+dss.getAverage());
```

Note that it is not too hard to write appropriate accumulator and combiner functions to compute these metrics by yourself.

Creating primitive streams ☝

Besides the static `of` and `generate` methods that can be used to create any of the three primitive streams using a fixed set of values or a generator function, `IntStream` and `LongStream` have two additional methods for creation of primitive streams:

1. `static IntStream range(int startInclusive, int endExclusive)`: Creates a sequential ordered `IntStream` from startInclusive (inclusive) to endExclusive (exclusive) by an incremental step of 1.
2. `static IntStream rangeClosed(int startInclusive, int endInclusive)`: Creates a sequential ordered `IntStream` from startInclusive (inclusive) to endInclusive (inclusive) by an incremental step of 1.

Another way to create primitive streams is to use the `mapToInt`, `mapToLong`, `mapToDouble` methods of the regular `Stream` interface. These methods take a `ToIntFunction`, `ToLongFunction`, and `ToDoubleFunction` respectively as an argument. For example: `DoubleStream ds1 = Stream.of(2, 3, 5, 9, 6).mapToDouble(i→i.doubleValue());` converts a `Stream` of Integers to a `DoubleStream`.

The primitive stream API too has a large number of classes and methods. But you don't need to memorize all of them. Just go through the JavaDoc descriptions of these classes to get a feel of how these classes work. If you see a method in the exam that you haven't read about or even seen before, apply common sense to guess what it does. The exam does not contain tricky questions on this topic.

21.2 Using Collectors ☝

The mutable reduction operation, which is done using `Stream`'s `collect` method, is by far the most commonly used operation of all stream operations. Although the `collect` method that you saw earlier offers a great deal of flexibility in implementation, it gets a bit too verbose. Specifically, since the supplier, the accumulator, and the combiner functions are passed to the `collect` method as three separate arguments, the call to `collect` can easily have a lot of repetition. For example, if I want to collect all the elements of a stream of strings into a `List`, my method call would look like this:

```
ArrayList<String> listOfWords = words.collect(ArrayList::new, ArrayList::add, ArrayList::addAll);
```

Since every invocation of `collect` invariably requires the three arguments (you can't omit any of the supplier, accumulator, and combiner functions), it would be a lot easier if the `collect` method accepted just one argument and retrieved the three functions by calling methods on that argument. That would also make it easy to organize the creation and management of the three functions.

This is exactly what the `Stream`'s `collect(Collector c)` method does. `java.util.stream.Collector` is an interface that acts as a repository of the three functions required by the mutable reduction operation. Instead of supplying the three functions separately, we ask the `collect` method to get the three functions from this repository. Not only does this approach reduces the verbosity of the method call, it also increases code maintainability by letting us put the code for the three functions into a separate reusable class.

But Java SDK doesn't stop at that. It also includes a utility class called `Collectors`, which contains implementations for a lot of commonly used mechanisms for mutable reductions. In other words, the `Collectors` class contains implementations for a lot of `Collector`s! You can get an instance of any of those `Collector` implementations by calling its static methods. Here is how the call to collect looks using this approach:

```
List<String> listOfWords = words.collect(Collectors.toList());
```

So, to reiterate, the `toList` method of the `Collectors` class returns a `Collector` that contains ready-to-use implementations of supplier, accumulator, and combiner functions that are suitable for "collecting" the elements of the stream in a `List`.

Here is another example. If you want to collect the words in a `Set` instead of a `List`, you can do `Set<String> setOfWords = words.collect(Collectors.toSet());`.
Although not important for the OCP Exam, you should know that besides providing supplier, accumulator, and combiner functions, a `Collector` may also optionally provide a "finisher" function. The job of this function is to perform the final transformation from the intermediate accumulation type to the final result type. This function is used when the type of the object returned by the supplier function is not the same as the type of the objects that the `Collector` is expected to return.

As of Java 21, the `Collectors` class contains implementations for more than 40 `Collector`s! While the OCP exam does not expect you to memorize all of them but it does expect you to know the various types of the `Collector`s and their design philosophy, which allows them to be chained together to create new functionality. In fact, one of the goals of the Collectors API was that writing and reading the code should feel as if you are expressing your idea in an English sentence. Trying to memorize the method names defeats the point of having such an expressive API. So, for example: when you see the statement `words.collect(Collectors.toList())`, you can easily guess that this statement is going to collect all the elements of the stream in a `List`. You don't really need to remember the minute details of the various functions that it uses internally. You can just expect it to do the right thing.

21.2.1 Collecting results into a Collection ☝

As you saw above, the `toList()` and `toSet()` methods collect the elements into a `List` and a `Set` respectively. However, the `Collector`s returned by these methods do not provide any guarantees on the type, mutability, serializability, or thread-safety of the `List`/`Set` that they use to collect the elements. There are also `toUnmodifiableList()` and `toUnmodifiableSet` methods that let you collect elements into an **unmodifiable** `List` and an **unmodifiable** `Set` respectively, but they throw a `NullPointerException` if the stream contains `null` values.

To get more control over the type of `Collection` in which you want to collect the elements, you can use the `toCollection(Supplier s)` method. For example:
`TreeSet<String> uniqueNames = namesStream.collect(Collectors.toCollection(TreeSet::new ));`

21.2.2 Collecting results into a Map ☝

You may use `toMap`, `toUnmodifiableMap`, and `toConcurrentMap` to collect the result into a `Map`. For example:

```
Stream<String> words = Stream.of( "hi", "salut", "hola", "namaste");
Map<String, Integer> map = words.collect(Collectors.toMap(word→word, word->word.length()));
System.out.println(map); //prints {hi=2, hola=4, namaste=7, salut=5}
```

Since a map stores key - value pairs, the `toMap` method needs at least two `Function`s - one to create a key and one to create the value that is to be associated with that key. Note that the lambda `word→word` implements a `Function` that returns the same value as the one that is passed to it. Such a function is called an identity function. You

may use `Function.identity()` method for this purpose as well. For example, you can invoke the `toMap` method in the above code like this: `toMap(Function.identity(), word->word.length())`. Looks more profound, doesn't it?

Normally, when you put a key-value pair in a `Map`, the existing value associated with that key in the map is replaced with the new value. However, the `toMap` method used above doesn't do that. It throws an `java.lang.IllegalStateException` (which is an unchecked exception) if the key function returns a key that is already present in the map. In other words, the two argument `toMap` method doesn't "merge" the new value silently. The `toMap` method expects you to provide a third function of type `BinaryOperator` to merge the values. For example:

```
Map<String, Integer> map = words.collect( Collectors.toMap(
  word→word.substring(0, 1),
  word→word.length(),
  (oldValue, newValue)→oldValue+newValue));
System.out.println(map);//prints {s=5, h=6, n=7}
```

Just like the `toCollection` method, if you want to have more control over the type of the `Map` that is used to collect the result, you can pass a `Supplier` that returns the `Map` object that you want used as the fourth argument.

21.2.3 Counting, averaging, and summing values ☝

There are several variants of the methods for counting, summing, and averaging the values of a stream. Since they follow the same pattern, you can easily understand them through a few examples. Assume the following definitions exist for each example.

```
record Student(String id, double gpa){};
```

```
Stream<Student> students = Stream.of(new Student("A1", 3.4), new
Student("A2", 4.0), new Student("A3", 3.1));
```

Counting elements

`long count = students.collect(Collectors.counting());` //returns 3
Observe that it returns a `long`.

Averaging values

There are three methods for calculating the average - `averagingInt`, `averagingLong`, and `averagingDouble`. Let's look at the `averagingDouble` method. It needs a `ToDoubleFunction` that can generate `double` values from the elements of the stream.

`double avgGpa = students.collect(Collectors.averagingDouble(s→s.gpa()));`
//returns 3.5

The `averagingInt` and `averagingLong` methods work the same way except that they take `ToIntFunction` and `ToLongFunction` as arguments respectively.

Summing values

Just like for computing average, there are three methods for summing up the values - `summingInt`, `summingLong`, and `summingDouble`. They too take `ToIntFunction`, `ToLongFunction`, and `ToDoubleFound` as arguments respectively. For example,

`double totalGpa = students.collect(Collectors.summingDouble(s→s.gpa()));`
//returns 10.5

Getting summary statistics

You have already seen classes such as `DoubleSummaryStatistics` for collecting summary statistics of primitive streams earlier. The same classes are used by the `Collector`s as well. But the methods are named `summarzingInt`, `summarizingLong`, and `summarizingDouble` and they take `ToIntFunction`,

`ToLongFunction`, and `ToDoubleFound` as arguments respectively. Here is an example:

```
IntSummaryStatistics iss = students.collect(Collectors.summarizingInt(s->(int)s.gpa()));
System.out.println(iss); //prints IntSummaryStatistics{count=3, sum=10, min=3, average=3.333333, max=4}
```

Find the maximum and minimum values

If you can create a `Comparator` that can compare the elements of a stream, you can use the `maxBy` and `minBy` methods to find the maximum and minimum values respectively.

```
Comparator<Student> c = (s1, s2)->{ return Double.compare(s1.gpa(), s2.gpa()); };
Optional<Student> topper = students.collect(Collectors.maxBy(c));
```

Observe that it returns an `Optional`. Recall that the `Stream` interface too has `max(Comparator c)` and `min(Comparator c)` methods. They too return an `Optional`.

21.2.4 Joining stream elements ☝

If you want to join the elements of a stream of `CharSequence`s (recall that `String` is a `CharSequence`) into one long `String`, there are three `Collector`s for this purpose:

1. `joining()` - Example:
   `String s = Stream.of("a", "b", "c").collect(Collectors.joining());` //returns "abc"

2. `joining(CharSequence delimiter)` - Example:
   `String s = Stream.of("a", "b", "c").collect(Collectors.joining(", "));` //returns "a, b, c"
   Observe that the delimiter is not appended after the last element.

3. `joining(CharSequence delimiter, CharSequence prefix, CharSequence suffix)` : - Example:
   `String s = Stream.of("a", "b", "c").collect(Collectors.joining(", ", "'", "'"));` //returns "'a, b, c'"
   Observe that the prefix and suffix are applied on the final joined string and not to each individual element.

Unfortunately, there is no `ToStringFunction` in the Java library and there is no method that can join elements of an arbitrary type into a string. So, you can't do something like: `students.collect(Collectors.joining(s → s.id()));`. You can, however, map the elements of a `Stream` to strings and then use one of the existing joining collectors. For example: `students.map(s → s.id()).collect(Collectors.joining(", "))`.

21.2.5 Partitioning and grouping stream elements ☝

Partitioning

You can separate the elements of a stream into two parts such that the elements in one part satisfy a `Predicate` and the elements in the other part don't. For example, the following code partitions a stream of students into two parts on the basis of their `gpa` :

```
Stream<Student> students = Stream.of(new Student("A1", 2.4), new
Student("A2", 4.0), new Student("A3", 3.1));
Map<Boolean, List<Student>> m =
```

```
students.collect(Collectors.partitioningBy(s -> s.gpa()>2.5));
out.print(m); //{false=[Student[id=A1, gpa=2.4]], true=
[Student[id=A2, gpa=4.0], Student[id=A3, gpa=3.1]]}
```

Observe that the collector returns a `Map` containing **exactly** two keys - `true` and `false`. The objects of the stream that satisfy the given `Predicate` are put in a `List` under `true` while the rest are put in a `List` under `false`. Note that if no elements satisfies the given predicate, the `List` mapped to `true` will be empty and all the elements will be in the `List` mapped to `false`. It will be the other way round if all the elements satisfy the given predicate.

Grouping

Grouping works the same way as partitioning except that instead of partitioning the elements into exactly two parts using a `Predicate`, `groupingBy` partitions the elements into one or more parts using a `Function`. For example, if you want to group students into three categories - below average, average, and above average, you can do it like this:

```
Map<String, List<Student>> m = students.collect(Collectors.groupingBy( s ->
s.gpa()<3 ? "below average" : s.gpa()⩾4 ? "above average" :
"average"));
System.out.println(m);
```

The above code prints, `{average=[Student[id=A3, gpa=3.1]], above average=[Student[id=A2, gpa=4.0]], below average=[Student[id=A1, gpa=2.4]]}`.

Observe that `groupingBy` too returns a `Map` but the keys of the map are the values returned by applying the `Function` to the stream elements. The keys are mapped to `List`s containing the elements of the stream.

Note that the `Collector` returned by the `partitioningBy` method doesn't accept `null` as a key. It throws a `NullPointerException` if the classifier `Function` returns `null`.

As usual, since the `groupingBy` method only promises to collect the results in a `Map`, there is no guarantee on the type, mutability, serializability, or thread-safety of the `Map` implementation returned by the collector. It is possible to tell the groupingBy collector to use a specific `Map` implementation class using an overloaded version of the groupingBy method but since that method also involves the concept of chaining downstream collectors, I will talk about it later.

21.2.6 Nesting collectors ☝

While going through the methods of the `Collectors` class, you will notice that it allows you to create `Collector`s for filtering and mapping the elements of a stream and also for reducing the elements of a stream through its `filtering`, `mapping`, and `reducing` methods. Recall that filtering and mapping are intermediate operations and can be performed using `Stream`'s `filter` and `map` methods while reduction is a terminal operation that can be performed using any of the `Stream`'s overloaded `reduce` methods. For example, I can reduce a stream of `Integer`s to the sum of their values using either of the following two statements:

```
Optional<Integer> sum = Stream.of(55, 65).collect(Collectors.reducing(Math::addExact));
Optional<Integer> sum = Stream.of(55, 65).reduce(Math::addExact);
```

One might wonder why the same functionality exists in `Stream` as well as in a `Collector`. Well, first of all, there is nothing wrong with `Stream`'s `reduce` method and, in fact, it is the preferred way if you want to reduce a stream directly. The same goes for Stream's `filter` and `map` methods. However, packaging these operations in a `Collector` allows the operation to be nested inside another `Collector`.

Let's dig a bit deeper into what nesting really implies here. Imagine a stream of water flowing from one point A to point B. We can say that from the perspective of point A,

point B is on the downstream. Similarly, from the perspective of a stream operation A, an operation B that is performed on the stream coming out of A, is the downstream operation. You have already seen this done in the examples presented earlier where we invoked multiple intermediate operations one after another in a stream pipeline and terminated the pipeline at the end with a terminal operation.

The `collect` method of `Stream`, however, is itself a terminal operation. There is never a stream coming out of the `collect` method. The stream is already terminated when the call to `collect` returns. So, how can another `Collector` be applied to the result produced by a `Collector`? It can't really be, at least not in the same way an intermediate operation is invoked after another.

The way the Collectors framework is designed is that a `Collector` can be created with a reference to a "downstream" `Collector`. To enable creation of a `Collector` with downstream `Collector`, many of the `xxxBy` methods of the `Collectors` class are overloaded to accept an extra parameter of type `Collector`. So, when a `Collector` is created with a reference to a downstream `Collector`, instead of returning the final result, the `Collector` passes on the results of the operation to that nested downstream `Collector` and lets that downstream `Collector` perform further reduction operations on that result. Since the downstream `Collector` is also created using one of the `xxxBy` methods of the `Collectors` class, it too may accept a `Collector` that will be applied further downstream. Thus, this nesting can go on as deep as you want. So, instead of chaining operations one after another externally, collectors let you nest an operation inside another.

Let's use this approach to modify a statement that I showed you earlier for joining `id` values of `Student` objects:

```
String str = students.collect(
  Collectors.mapping( s→s.id(), Collectors.joining() )
);
```

The above statement creates a `mapping` collector with a nested `joining` collector. The `mapping` collector will map `Student` objects to `String` objects and then pass

the resulting stream of strings to the `joining` collector. The `joining` collector joins all the incoming strings into one long string.

Although we didn't gain much here by using this approach, but what if we want to count the number of students who passed or failed? This requires us to first partition the stream on incoming students into two lists (this can be done by the partitioningBy collector) and then find out the size of each list by iterating through the map. Something like this:

```
Map<Boolean, List<Student>> counts = students.collect(Collectors.partitioningBy(s→s.gpa()>2.5));
for(Map.Entry<Boolean, List<Student>> me : counts.entrySet()){
  System.out.println(me.getKey()+" = "+me.getValue().size());
}
```

Now, check out the same code using a nested collector:

```
Map<Boolean, Long> counts = students.collect(Collectors.partitioningBy(s->s.gpa()>2.5, Collectors.counting()));
System.out.println(counts);
```

Isn't that better? Since there is no limit to nesting, you just need to think about the how you want to build a stream pipeline logically and then use appropriate nested collectors to achieve the same. Here is another example. The following code partition the incoming students into two categories - the ones who passed and the ones who failed, and then joins the ids of the students in those two categories:

```
Stream<Student> students = Stream.of(new Student("A1", 2.4), new Student("A2", 4.0), new Student("A3", 3.1));
Map<Boolean, String> ids = students.collect(
  Collectors.partitioningBy(
    s→s.gpa()>2.5,
    Collectors.mapping(
```

```
        s->s.id(),
        Collectors.joining(", "))));
System.out.println(ids);
```

This prints: `{false=A1, true=A2, A3}`.

The above code creates a partitioning collector with a nested `mapping` collector with a nested `joining` collector. The partitioning collector partitions incoming stream of students into a `Map` containing two `List`s, but instead of returning that map, the partitioning collector uses the nested `mapping` collector to convert those lists of students into lists of strings. Lastly, the `mapping` collector uses the nested `joining` collector to join each of the lists of strings into single strings. You can try accomplishing the same without using collectors and you will realize that this approach is a lot shorter. Professional code often uses `import static java.util.stream.Collectors.*;` and that shortens the code even more.

Although this approach takes some time to get used to, but once you get the hang of it, it feels very natural.

The exam does not expect you to remember which `Collector` accepts a downstream `Collector` and which does not. It does not trick you with that trivia. If you see a method invocation in the exam that takes a downstream collector, you can assume that it does take a downstream collector. The exam focuses more on what that method invocation does rather than on its legal correctness.

In general though, you can use the following trick to figure out if a `Collector` accepts a downstream `Collector`. Any operation that transforms a stream to another stream or reduces it to a collection of some sort accepts a downstream `Collector`. For example, collectors returned by `filtering`, `mapping`, `groupingBy`, and `partitioningBy` methods accept a downstream `Collector` while collectors returned by `counting`, `reducing`, `joining`, `maxBy`, `minBy`, summingXXX, averagingXXX, and summarizingXXX methods do not. An exception to this trick is that `toList` and `toMap` (and other similar

methods) do not accept a downstream collector.

21.2.7 Identifying the input type and the return type while using collectors ☝

The Collectors API relies heavily on generics as it inferences types from the context. You may go through the Collections and Generics chapter to refresh the basics of generics before proceeding further.

Let's start with the `Stream` interface. It declares a type parameter `T`, which represents the type of the objects this stream is going to process. Next, check out the declaration of `Stream`'s `collect` method: `<R,A> R collect(Collector<? super T,A,R> collector)`. This method declares two more types parameters named `R` and `A` and it also uses `R` as the return type of the method. At this point, the `collect` method has access to three type parameters - `T`, `A`, and `R`, where `T` is accessible to the whole of the `Stream` interface while `A` and `R` are accessible only in the `collect` method.

Now, check out the definition of the `Collector` interface. `Collector` too is a generic interface that deals with objects of three types. These three types are represented by the three type parameters named `T`, `A`, and `R` that it declares. Note that these three type parameter names have no relation to the three type parameters declared in the `collect` method yet because `Collector` and `Stream` are two different interfaces and therefore, these type parameters belong in two different scopes. (It is like two classes having a variable with the same name. They have no relation.)

Coming back to `Stream`'s `collect` method, observe the type of the method parameter named `collector`. The type of this method parameter is a `Collector` that is typed to the three type variables that are accessible here, i.e., `? super T`, `A`, and `R`. In other words, the type parameters `T`, `A`, and `R` of the `Collector` interface are being typed to `? super T`, `A`, and `R` type parameters of the

`Stream` interface by this method. This is what establishes the relation between the type parameters of the `Stream` interface and the type parameters of the `Collector` interface. This implies three things:

1. The `collect` method can only accept a `Collector` that is capable of consuming objects of type `T` or of any super type of `T`.
2. The type of the value returned by the `collect` method is the same as the third type parameter `R` of the `Collector` that is being passed the `collect` method.
3. There is no restriction on `A`.

Thus, to check whether a particular invocation of the `collect` method is valid or not, you need to check the types of `T` and the `R` of the `Collector` that is returned by the method of the `Collectors` class in the JavaDoc. For example, if you look at the declaration of `Collectors.counting()` method in JavaDoc, you will notice that it returns `Collector<T,?,Long>`. This means that this collector can work with whatever is the type of the incoming objects (represented by `T`), it doesn't reveal the type of `A` that it uses (`?`), and it will always return a `Long` because it types `R` to `Long`. Thus, both of the following statements are valid:

```
Long count = students.collect(Collectors.counting());
Long count = Stream.of(1, 2, 3).collect(Collectors.counting());
```

Now, take a look at `Collectors.joining()`. It is returns `Collector<CharSequence,?,String>`. This means, it can only work when the incoming elements are of type `CharSequence` and it will always return a `String`. Thus, the following statement will not compile because the incoming stream is of `Integer`s:

```
String str = Stream.of(1, 2, 3).collect(Collectors.joining());//will not compile
```

Let's look at one more before moving on to a more complicated one. The `maxBy` method returns `Collector<T,?,Optional<T>>`. It imposes no restriction on `T`, and so, it can work with any kind of stream. It will always return an `Optional` of whatever is the type of that stream. Of course, the declaration also says that it takes

a method parameter of type `Comparator<? super T>`, which means we must pass a `Comparator` that can compare the elements of the incoming stream.

Now, consider the following statement:

```
students.collect(partitioningBy(s -> s.gpa()>2.4, mapping( s -> s.id(), joining() )));
```

Let's figure out what it returns. Here, we have a `patitioningBy` collector that nests a `mappingBy` collector that nests a `joining` collector. The `partitioningBy` method is declared as:

```
Collector<T, ?, Map<Boolean,D>> partitioningBy(Predicate<? super T> predicate, Collector<? super T,A,D> downstream)
```

As explained earlier, a `partitioningBy` collector that has no downstream collector partitions the stream based on a `Predicate` and returns a `Map` of two key-value pairs where keys are `true` and `false` and the values are `List`s of elements of the stream. However, a `partitioningBy` collector that has a nested downstream collector doesn't decide the values associated with the two keys. It simply splits the incoming stream into two streams (one that contains objects that satisfy the given `Predicate` and one that don't), passes those streams to the downstream collector separately and uses the values returned by the downstream collector for each of those streams to populate the map. Thus, if the downstream collector returns a result of type `D`, the partitioning collector will return a `Map<Boolean, D>` object as its result instead of `Map<Boolean, List<T>>`. Note that the downstream collector is invoked for each of the two streams individually. How do I know it does that? Through the JavaDoc of this method. Okay, so, we now know that the input to the downstream collector in this case is `Stream<Student>`. Let's look at the mapping collector. It is declared as:

```
Collector<T,?,R> mapping(Function<? super T,? extends U> mapper, Collector<? super U,A,R> downstream)
```

The mapping collector converts the elements of type `T` (i.e. `Student` in this case) to

elements of type `U`. The type of `U` is determined by the mapper function. Here, the mapper function is `s→s.id()`. Since `s.id()` returns a `String`, `U` is being typed to `String` here. Thus, the mapping collector used here converts a stream of students into a stream of strings and, again, instead of returning the result immediately, passes this new stream to the collector further downstream, i.e., the joining collector to generate the result. You have already seen how the joining collector works. It takes a stream of `CharSequence`s and joins them into one long `String`. There is no further nested collector and the result of the joining collector is returned to the caller, i.e., to the mapping collector. The mapping collector returns that string to the partitioning collector, which puts it into the map. This is the map that the partitioning collector finally returns. Thus, you can see that that the above statements returns a `Map<Boolean, String>`.

You could do the same thing by creating variables for the nested collectors separately and by using those variables in various method calls as follows:

```
Collector<CharSequence, ?, String> jc = Collectors.joining();
Collector<Student, ?, String> mc = Collectors.mapping(s→s.id(), jc);
Collector<Student, ?, Map<Boolean, String>> pc = Collectors.partitioningBy(s-
>s.gpa()>2.4, mc);
Map<Boolean, String> result = students.collect(pc);
```

Observe that the inner most collector needs to be created first.

21.2.8 Using a teeing Collector ☝

Recall that a stream cannot be "restarted" once it ends. So, if you want to process a stream in two different ways it is not possible to reuse the same stream after processing it in one way. You have to create the stream from the source again. If that is not feasible, then one option is to collect the elements of the stream into a collection

and then process it in multiple ways. Note that you can add multiple stream operations to a stream pipeline but if an operation alters the stream, such as by filtering or mapping, then the other operation will not be able to see the original elements of the stream.

With Java 12, there is another option. The `Collectors` class offers a teeing collector that allows you to attach two stream pipelines (in the form of two downstream collectors) to a single stream pipeline. The teeing `Collector` passes incoming stream elements to both the downstream collectors. When the downstream collectors are done, it merges the result of the two collectors into a single result object and returns that result object. The method to get the teeing collector is declared as follows:

```
static <T,R1,R2,R> Collector<T,?,R> teeing(Collector<? super T,?,R1> downstream1, Collector<? super T,?,R2> downstream2, BiFunction<? super R1,? super R2,R> merger)
```

Here is an example that uses this collector:

```
Stream<Character> stream = Stream.of('a', 'b', 'c', 'd');
Collector<Character, ?, String> sc = Collectors.mapping(ch→""+ch,
Collectors.joining());
Collector<Character, ?, Long> cc = Collectors.counting();
String result = stream.collect(
  Collectors.teeing(sc, cc, (word, length)→word+" "+length));
System.out.println(result); //prints abcd 4
```

21.2.9 Quiz ☝

Q. Given:

```
record Account(int id, String type, double balance){};
```

and

```
Stream<Account> sa = Stream.of(new Account(1,"CHK", 1000.0), new Account(2,"CHK", -500.0), new Account(3, "SAV", 400.0), new Account(4,"FD", 9000.0));
```

What will be the return type of the following statement?

```
sa.collect(partitioningBy( a→a.balance()>0 , groupingBy(a→a.type(), counting())));
```

A. `List<Long>`

B. `Map<String, Long>`

C. `Map<Boolean, Map<String, Long>>`

D. `Map<Boolean, Long>`

E. `Long`

The correct answer is **C**.
The partitioning collector partitions the incoming stream into two streams based on the given `Predicate` and passes those streams to the downstream collector individually. Thus, we can be sure that the return type will be a `Map` whose keys are `Boolean` but the type of the values will be decided by the downstream collector. The downstream collector here is a grouping collector that splits an incoming stream into multiple streams based on the value returned by the given grouping `Function` and passes the resulting streams to its downstream collector. Thus, we can be sure that the return type of this collector will be a `Map` whose keys are `String` (because the given grouping function returns a `String`) and the type of values will be determined by the collector further downstream. The collector further downstream is a counting collector, which simply counts the number of objects in the incoming stream and returns a `Long`. Thus, the return type of the grouping collector will be `Map<String, Long>` and the return type of the partitioning collector will be `Map<Boolean, Map<String, Long>>`.

You should run the code and verify the output.

21.3 Advanced stream concepts ☝

21.3.1 Ordering ☝

As mentioned before, a stream takes its elements from a source. The source can be a collection, an array, or a generator function. Recall that when you iterate through an array or a `List`, you are guaranteed to get elements in the same order (aka called encounter order). If the source guarantees to provide elements to the stream in a deterministic order, the result of a stream operation will also be in that order. So, for example, in the following code, the order of the elements in the resulting list will always be the same as the order of the elements in the source stream, i.e., `2`, `1`, `3` because a `List` (and all `SequencedCollection`s, since Java 21) does define an encounter order:

```
List.of(2, 1, 3).stream().collect(Collectors.toList());
```

Similarly, `Stream.of(2, 1, 3).collect(Collectors.toList());` too preserves the order because it creates a stream using an array, which is also intrinsically ordered.

There are a few exceptions to this rule though. These are as follows:

1. If the source of the stream reports that it is unordered then the stream pipeline is free to optimize its execution without worrying about the order of the

1. incoming elements.
   Note that there is no method in `Stream` to check whether it is ordered or not because the ordering really depends on the source. So, to determine whether a stream is ordered or not, you have to check whether the stream source is ordered or not. For example, to check whether a given collection is ordered or not, you can do: `collection.spliterator().hasCharacteristics(Spliterator.ORDERED)`. This will return `true` for an `ArrayList` and a `TreeSet` but `false` for a `HashSet`. Thus, a stream created from an `ArrayList` is ordered but from a `HashSet` is not.
2. A stream created using a generator function (i.e. using the `Stream.generate(Suppliers)` method) is always unordered. This is mandated by the specification of the `Stream`'s `generate` method.
3. A stream operation may impose its own order on the elements, in which case, the original order, if there was any, will be lost. For example, the `sort` operation sorts the incoming elements. Therefore, the outgoing stream from the `sort` operation will have a new order of elements.
4. As of Java 21, two of the stream operations are explicitly defined to be "non-deterministic". Meaning, they don't care about the order of the incoming elements and may process the elements in any order. These are `forEach(Consumer c)` and `findAny()`. Thus, there is no guarantee that `Stream.of(1, 2, 3).forEach(System.out::println);` will print the numbers `1`, `2`, and `3` in that order or that `Stream.of(1, 2, 3).findAny()` will always return a particular value. There is a `forEachOrdered` operation in Stream that can be used to process the elements in the same order though. So, `Stream.of(1, 2, 3).forEachOrdered(System.out::println);` is guaranteed to print the numbers `1`, `2`, and `3` in that order.
5. If you tell the stream to ignore the order by adding the `unordered()` operation in the pipeline, the stream pipeline is free to ignore the order. This operation does not actively disturb the order of the elements but just lets the stream pipeline ignore the order if it wants to. You can't restore a stream order once it is unordered.

I haven't talked about parallel stream yet but removing the ordered constraint from a stream helps increase the performance when the stream is execution is parallelized. For example, if you are just summing a stream of Integers, you don't really care about the order in which the values are summed. Thus, `aLongListOfInts.stream().unordered().collect(Collectors.summingInt(i->i));` can perform better when parallelized.

However, be aware that unordering a stream may also have unintended consequences. For example, there is no guarantee that `aLongListOfInts.stream().unordered().parallel().limit(3).forEachOrdered(System.out::println );` will always print the first three values of the input list or that `aLongListOfInts.stream().unordered().parallel().findFirst()` will always return the first element.

21.3.2 Quiz ☝

Q. Given:

```
List<String> los = List.of("", "", "a", "", "b");
Set<String> sos = Set.of("", "a", "b");

List<String> result1 = los.stream().dropWhile(s→s.isEmpty()).toList();
System.out.println(result1); //1
List<String> result2 = sos.stream().dropWhile(s→s.isEmpty()).toList();
System.out.println(result2); //2
```

Identify **3** correct statements:

A. //1 will **always** print `[a, , b]`.
B. //1 may print `[ ]`.

C. //2 may print `[ ]`.
D. //2 may print `[b, , a]`.
E. //2 will **never** print `[ , a, b]`.

Correct answer is **A, D, E**.
The `dropWhile(Predicate p)` method drops the elements of a stream until it gets an element that fails the predicate. In this case, the predicate tests whether the string is empty or not. Therefore, it will drop the elements from the beginning until it reaches a non-empty string. Once it finds a non-empty string element, it will pass on the remaining elements to the next operation in the pipeline. Now, remember that `List` is ordered while `Set` is not. Therefore, the stream at //1 will always encounter elements in the order of the elements in the source list, while the stream at //2 may encounter the elements in any order. If the first element that the stream at //2 encounters is not an empty string, the predicate will fail and all of the elements will be in the resulting stream. Therefore, options **A** and **D** are correct. **B** and **C** are incorrect because if the first element in the stream is an empty string, it will be dropped, which means, the first element of the resulting stream can never be an empty string. For the same reason, option **E** is correct.

21.3.3 Parallel streams ☝

I believe the quiz in the "Using Collectors" section, in which a multi-map was created in a single line of code, has convinced you of the power of the Stream API. But the benefit of reduction in the size of the source code is nothing as compared to the benefit that streams provide in terms of parallelizing the execution of the business logic without writing a single line of multi-threading code.

By default, streams are **sequential**, which means that the stream framework uses only one thread to execute a stream pipeline. This is the thread that picks up elements from the source, executes the functions supplied to the intermediate operations, and then executes the terminal operation. It does everything. This is evident from the output of the following code:

```
Stream.of(1, 2, 3).map(i -> {
  out.println("T " + Thread.currentThread().threadId() + " doubling " + i);
  return i * 2;
}).map(i -> {
  out.println("T " + Thread.currentThread().threadId() + " mapping " + i);
  return "" + i;
}).forEach(i -> out.println("T " + Thread.currentThread().threadId() + "
printing " + i));
```

The above has two mapping operations and a forEach operation in the stream pipeline. It prints:

```
T 1 doubling 1
T 1 mapping 2
T 1 printing 2
T 1 doubling 2
T 1 mapping 4
T 1 printing 4
T 1 doubling 3
T 1 mapping 6
T 1 printing 6
```

Observe that the same thread id is printed in all of the print statements. This is fine for trivial code like above but what if the stream had a few thousand values and the mapping operations contained long running computations? It would take forever to complete! This is precisely the reason why programmers use multithreading. As you saw in the Concurrency chapter, multithreading allows tasks to be processed in parallel, which improves overall throughput by taking advantage of multiple CPU cores and also by efficiently utilizing system resources. But in spite of the availability of high level APIs in Java SDK, writing multithreaded code is still a lot of work. With streams, all you have to do to achieve the same is to just say the word parallel:

```
Stream.of(1, 2, 3).parallel().map(i -> {
  out.println("T " + Thread.currentThread().threadId() + " doubling " + i);
  return i * 2;
}).map(i -> {
  out.println("T " + Thread.currentThread().threadId() + " mapping " + i);
  return "" + i;
}).forEach(i -> out.println("T " + Thread.currentThread().threadId() + "
printing " + i));
```

The only thing different in the above code is that I have inserted the `parallel()` operation in the pipeline. It produces the following output:

```
T 21 doubling 1
T 22 doubling 3
T 21 mapping 2
T 22 mapping 6
T 21 printing 2
T 22 printing 6
T 1 doubling 2
T 1 mapping 4
T 1 printing 4
```

As you can see, multiple threads were utilized by the stream pipeline to process the elements.

Due to the declarative way of creating a stream pipeline, the underlying stream framework is capable of executing operations in parallel transparently after taking into account the availability of the system resources. But to let the stream framework achieve this performance boost without jeopardizing thread safety, the programmer must be aware of certain concepts and must follow certain practices while creating a stream pipeline. In particular, one must know the answers to the following questions to be able to parallelize stream pipeline execution correctly:

1. How do you make a stream parallel?
2. How many threads does a parallel stream use?
3. How is the processing split among multiple threads?
4. What happens to the order of the elements?
5. What happens when stream operations are stateful?
6. What happens when stateful functions are passed as behavioral arguments?
7. How is the result of a reduction operation produced while using a parallel stream?
8. How is thread safety ensured while using mutable reduction operations?

Let's go over the above points one by one.

Creating parallel streams ☝

The `parallel()` and `sequential()` intermediate operations

The easiest way to create a parallel stream is to add the intermediate operation `parallel()` in a the stream pipeline. It doesn't matter where in the pipeline it is added as long it is added before the terminal operation. The place where it is added is not important because the call to `parallel()` actually doesn't do anything except set a flag that tells the stream framework to process the stream pipeline in parallel. A stream is either parallel or it is not and all intermediate operations, whether they appear before or after the call to `parallel()` are affected by this setting.

You can check if a stream is parallel or not by calling the `isParallel()` method on the `Stream` and if you don't want a stream to be processed in parallel (you will see shortly that there are genuine reasons for not wanting to process a stream in parallel), you can add the intermediate operation `sequential()` to the stream pipeline. It doesn't matter how many times you add `parallel()` and `sequential()` operations in the stream pipeline. The fate of the stream is decided by the last operation that you add because, as I mentioned before, these operations just set or unset an internal flag.

The `parallelStream()` method

Besides the `stream()` method, the `Collection` interface also has a default method named `parallelStream()` that returns a parallel stream. This method is primarily

meant for the developer of a `Collection` class wherein they can customize the splitting of the contents of the collection.

You will see both the methods used in the exam. There is no difference between the two methods from the perspective of the end result.

Number of threads used by a parallel stream ☝

The stream frameworks uses an optimum number of threads based on the number of processing cores available on the machine. This number cannot be customized using the Stream API but can be configured using a command line switch while starting the JVM. Since this is way beyond the scope of the exam, I will leave it at that.

Splitting of stream elements among threads ☝

The stream framework uses the `Spliterator` provided by the underlying stream source to split the elements into multiple groups as it deems fits. In general, you should not make any assumption on how the elements are split.

Ordering of elements in a parallel stream ☝

For an ordered stream, both sequential and parallel execution of a stream pipeline are guaranteed to produce the same end result. However, there is no guarantee on the order of execution of individual stream operations in the pipeline. What this means is that multiple threads may execute a particular stream operation for a particular stream element in any order as long as it is able to produce the same end result. The parallel stream pipeline will combine the results of its execution such that the end result will be the same as the result produced by a sequential stream. Consider the following code:

```
Stream<Integer> strm = Stream.of(1, 2, 3, 4, 5).parallel();
strm
  .peek(i→{ System.out.println("Peeking before map "+i);} )
  .map(i→{ System.out.println("Mapping "+i); return i+1; } )
```

```
    .peek(i->{ System.out.println("Peeking after map "+i);} )
    .filter(i->{System.out.println("Filtering "+i); return i%2==0;} )
    .forEachOrdered(i->{ System.out.println("Printing final "+i);} );
```

It produces the following output:

```
Peeking before map 1
Mapping 1
Peeking after map 2
Peeking before map 3
Peeking before map 5
Mapping 5
Peeking after map 6
Filtering 6
Peeking before map 2
Peeking before map 4
Mapping 4
Peeking after map 5
```

```
Filtering 5
Filtering 2
Mapping 3
Mapping 2
Printing final 2
Peeking after map 4
Filtering 4
Peeking after map 3
Filtering 3
Printing final 4
Printing final 6
```

You may get different output if you run the code multiple times and you will notice that the output from print statements in the intermediate operations does not follow any particular order. This is expected and is fine because we are, after all, executing the operations in parallel. However, since the stream is ordered and since the terminal operation too is ordered, the output from print statement in the `forEachOrdered` operation will always be in the encounter order of the stream source. This order would not be guaranteed if the stream is unordered or if the terminal operation itself is non-deterministic (for example, `forEach` or `findAny`).

Processing a stream pipeline while respecting the order of elements in the stream is quite an onerous responsibility and it can reduce the performance of a parallel stream pipeline if the pipeline contains operations such as `limit` or `findFirst` that force

the pipeline to reorganize the results of intermediate operations in the original order of the stream elements. Again, since exam does not focus on performance, I am not going into the details of this aspect but I encourage you to read more on this topic online. The JavaDoc description given for the `java.util.stream` package is a must read.

Using stateful operations with parallel streams ☝

As explained before stateful operations need to maintain a state that depends on the elements passing through the stream. While many stateful operations such as `distinct` do not need to wait for the stream to end before generating the outgoing stream some such as `sort` do. In either case, a stateful operation reads as well as updates its internal state as new elements pass through it. You can now imagine how this will play out when the stream processing is parallelized. Maintaining a state that is shared with multiple threads needs synchronization. The good news is that you need not worry about it. Whatever synchronization is needed by a stateful operation is managed by the stream framework. The bad news is that synchronization induces a performance penalty. Furthermore, since the stream processing must respect the order of elements if the stream is ordered, many seemingly innocuous stateful operations such as `dropWhile`, `takeWhile`, `limit` and `skip` perform poorly when an ordered stream is parallelized.

So, while making a stream parallel is easy, making the decision to make a stream parallel is not so easy. There is no universally accepted formula that can help you decide. The best way to make this decision is to actually benchmark your application with realistic data and see whether making the stream parallel improves performance or not.

Using stateful behavioral arguments with parallel streams ☝

Passing a stateful function to a stream operation suffers from all of the issues described above, and on top them, the function is responsible for the thread safety of the state that it keeps. For example, assuming that sum is a shared int variable, there

is no guarantee that the following code will correctly calculate the sum: `IntStream.range(0, 10000).parallel().forEach(i→{ sum = sum+i; });`. The lambda `i→{ sum = sum +i; }` is a stateful lambda and is shared by multiple threads but it does not synchronize access to `sum`. Of course, if you synchronize the summation, the benefits of parallelism will be lost.

Reducing a parallel stream ☝

Since a non-mutable reduction creates a new result value after processing each element, it does not have any shared state to manage. Thus, as long as the accumulator and the combiner functions do not have a state of their own (because they may get invoked from multiple threads simultaneously) and as long as they don't try to modify the arguments they are passed, you do not need to worry about the thread safety of a non-mutable reduction. For example, the following code is completely thread safe:

```
//defining this method separately to conserve space
static long tid(){ return Thread.currentThread().threadId(); }

//in main
String result = Stream.of("a", "b", "c", "d").parallel().reduce("",
  (partialResult, s)→{
    System.out.println("T "+tid()+" Accumulating -
partialResult="+partialResult+" newvalue="+s);
    return partialResult+s;
  },
  (partialResult1, partialResult2)→{
    System.out.println("T "+tid()+" Combining -
partialResult1="+partialResult1+" partialResult2="+partialResult2);
    return partialResult1+partialResult2;
  });
```

```
System.out.println(result);
```

It produces the following output:

```
T 21 Accumulating - partialResult= newvalue=b
T 1 Accumulating - partialResult= newvalue=c
T 22 Accumulating - partialResult= newvalue=a
T 23 Accumulating - partialResult= newvalue=d
T 22 Combining - partialResult1=a partialResult2=b
T 23 Combining - partialResult1=c partialResult2=d
T 23 Combining - partialResult1=ab partialResult2=cd
abcd
```

Observe the following points in the above output:

1. Four threads were used to accumulate the stream elements with the identity value in parallel. Partial results were also combined in parallel using different threads.
2. The order in which the accumulation and the combining is done is not the same as the order of the elements in the stream. Even so, the order in the final result was the same because the stream was ordered.
3. The accumulator and combiner functions do not have any state and do not modify the arguments.

Identity value while reducing parallel streams ☝

The value passed for the identity parameter is crucial for maintaining the consistency of the result generated by the reduce methods. Since a stream may be split into multiple fragments and each fragment may be reduced independently, the identity value may be used an **unpredictable** number of times while reducing those fragments. Thus, the identity value should be chosen such that accumulating the identity value with any element produces a value that is the same as the value of the element. In other words, `accumulator.apply(identity, t)` should always return the same value as `t`. For example, `0` is an appropriate identity value if the accumulator function adds the two values while `1` is appropriate if it multiplies them.

Not adhering to this rule will produce inconsistent results. For example, `Stream.of(1, 2, 3, 4).parallel().reduce(1, (a, b)→a+b);` will return `14` if the stream is split into four fragments but since there is no guarantee that the stream will always be split into four fragments, the result may not always be `14`.

For the same reason, identity should ideally be immutable or, at least, the accumulator function must not modify the object passed as identity because doing so will affectively cause different stream fragments to receive a different identity value when processed in parallel.

Mutable reduction of parallel streams ☝

Since a mutable reduction mutates the same result object every time it processes an element of the stream, one may think that a mutable reduction needs a lot of work to make it thread safe. But it is not so. The stream framework ensures that a mutable reduction is thread safe even when the result container is not thread safe. However, the way thread safety is managed in a mutable reduction is very different from the way it is managed in a non-mutable reduction. While performing mutable reduction, instead of creating a single result container object and synchronizing access to it, the stream framework creates multiple result container objects - one for each fragment into which a stream is split. It further ensures that each mutable result container object is accessed from exactly one thread at a time thereby eliminating the need for synchronization. At the end, it merges multiple result container objects into a single object (using the combiner function) and returns that object. The following example makes this clear:

```
Supplier<List<String>> supplier = ()→{
  ArrayList<String> al = new ArrayList();
  System.out.println("Supplier invoked by thread "+tid()+" AL =
"+System.identityHashCode(al));
  return al;
};
```

```
BiConsumer<List<String>, Integer> accumulator = (l, i)->{
  System.out.println("Accumulator invoked by thread "+tid()+"
AL="+System.identityHashCode(l)+" Value:"+i);
  l.add(""+i);
};

BiConsumer<List<String>, List<String>> combiner = (l1, l2)->{
  System.out.println("Comibiner invoked by thread "+tid()+"
L1="+System.identityHashCode(l1)+" L2:"+System.identityHashCode(l2));
  l1.addAll(l2);
};

List<String> result = List.of(1, 2, 3, 4).stream().parallel().collect(supplier,
accumulator, combiner);
System.out.println(result);
```

The above code collects `Integer`s from the incoming stream into a `List` of `String`s. Note that `ArrayList` is being used as the mutable result container here even though it not a thread safe class. But above code works fine. It produces the following output:

```
Supplier invoked by thread 21 AL = 26605576
Supplier invoked by thread 23 AL = 1117808977
Supplier invoked by thread 1 AL = 834600351
Supplier invoked by thread 22 AL = 247510350
Accumulator invoked by thread 21 AL=26605576 Value:2
Accumulator invoked by thread 22 AL=247510350 Value:1
Accumulator invoked by thread 1 AL=834600351 Value:3
Accumulator invoked by thread 23 AL=1117808977 Value:4
Comibiner invoked by thread 1 L1=834600351 L2:1117808977
Comibiner invoked by thread 21 L1=247510350 L2:26605576
Comibiner invoked by thread 21 L1=247510350 L2:834600351
```

```
[1, 2, 3, 4]
```

The following points are evident from the above output:

1. The stream was split into four fragments and the `Supplier` function was invoked for each of the fragments.
2. Different threads got different `ArrayList` objects and the accumulator function was executed in parallel using different `ArrayList` objects. Partial results were also combined in parallel using different threads.
3. The order in which the accumulation and the combining is done is not the same as the order of the elements in the stream. Even so, the order in the final result was the same because the stream was ordered.
4. The accumulator and the combiner functions do not have any state but they do modify the result container.

Thus, the key to ensure thread safety of a mutable reduction is to use a `Supplier` function that returns a new mutable result container and the rest is taken care of by the stream framework.

Thread safety while using a Collector ☝

Performing mutable reduction using any of the readymade collectors provided by the `Collectors` class is also thread safe. You do not need to write any code for ensuring the thread safety of the operation. So, for example, the following code is thread safe:

```
List<String> result = List.of(1, 2, 3, 4).stream().parallel().map(i->""+i).collect(Collectors.toList());
```

But again, if you pass stateful lambdas as arguments to stream operations, you are responsible for the thread safety of that state.

Using concurrent collectors ☝

Achieving thread safety of mutable reduction using multiple instances of thread-

unsafe result containers is a neat trick but it has one issue - the contents of the container instances need to be merged into one final result object at the end. Depending on what kind of container is being used, the merge operation may be expensive. For example, if your mutable result container is a `Map`, merging multiples maps is an expensive operation.

Well, there is a solution to that as well - use concurrent collections. Note that I am not talking about synchronized collections, which you can create using synchronizedXXX methods of the `Collections` class, because they synchronize each operation and that will nullify the whole point of making the stream parallel. I am talking about concurrent collections such as `java.util.concurrent.ConcurrentHashMap`. These classes provide better performance than synchronized collections by allowing concurrent operations. So, instead of creating new instances of the result container class in your `Supplier` function, you can use the same instance of a class that supports concurrent operations. A mutable reduction using concurrent collections is called "**concurrent reduction**".

The `Collectors` class provides two methods - `toConcurrentMap` and `groupingByConcurrent` that work on this principle. For example, the statement `Map<Boolean, List<Integer>> result = List.of(1, 2, 3, 4).stream().parallel().collect(Collectors.groupingByConcurrent(i -> i%2==0, Collectors.toList()));` uses a concurrent map to group the incoming values. The nested `Collector` returned by `toList` is a regular collector though.

Although the exam does not go too deep into concurrent reduction, you should be aware that since multiple threads can accumulate results into a shared concurrent container any time, the order in which results are accumulated cannot be deterministic. Therefore, a concurrent reduction for a parallel stream is only possible if - **1.** the collector has `CONCURRENT` characteristic and **2.** the stream is unordered or the `Collector` explicitly says that it ignores encounter order by having `Collector.Characteristics.UNORDERED` characteristic.

21.3.4 Quiz ☝️

Q. Identify correct statements about the following code:

```
Optional<String> result = Stream.of(1, 2, 3, 4)
  .parallel().map(i -> {
    System.out.println("Mapping "+i);
    return "" + i;
}).findFirst();
System.out.println(result);
```

Select **3** correct options.

A. It will always print `Optional[1]`.
B. It may not always print `Optional[1]` if `unordered()` operation is **added** to the stream pipeline.
C. It will always print `Optional[1]` if `parallel()` is **removed** from the pipeline.
D. It will always print `Mapping 1` to `Mapping 4` in that order.
E. It will always print `Mapping 1` to `Mapping 4` in that order if `sequential()` operation is added to the stream pipeline **instead of** `parallel()`.
F. It may print `Mapping 1` to `Mapping 4` in any order **only if** `unordered()` operation is added to the stream pipeline.

Correct answer is **A, B, C**.

Observe that the stream is ordered because it is being created from an ordered source (an array) and the given stream pipeline does not have an `unordered()` operation to make it unordered. For an ordered stream, the end result will always be in accordance with the order of the elements in the stream, irrespective of whether the stream is sequential or parallel. Since `findFirst()` returns the first element, it will always return `Optional[1]` in this case. Hence, **A** and **C** are correct.
The `unordered()` operation makes the stream unordered, thereby allowing the stream pipeline to disregard the order of the elements. In an unordered stream,

`findFirst()` is allowed to return any element irrespective of whether the stream is sequential or parallel. Hence, **B** is correct.
The execution order of intermediate operations is not guaranteed even for ordered streams when the stream is parallel. Since the given stream is parallel, the order in which the mapping operation will be invoked for the elements is not guaranteed. Therefore, **D** and **F** are incorrect.
If the stream is made sequential instead of parallel, each element will be processed by a single thread, one at a time. However, the `findFirst()` operation will short circuit the stream pipeline after returning the first element, leaving the rest of the elements unprocessed. Thus, the `map()` operation will be executed only for the first element. Therefore, making the given stream sequential will cause the program to print only `Mapping 1` instead of `Mapping 1` to `Mapping 4` and so, **E** is incorrect.

21.3.5 Parallel Streams and Virtual Threads ☝

The stream framework uses a common pool of threads maintained by the JVM called the common ForkJoin Pool to execute stream operations in parallel. Don't worry, ForkJoin is not on the exam, so, this section is just FYI. The threads in this common ForkJoinPool are platform threads, which means a parallel stream uses platform threads to execute the stream pipeline. However, what if the stream itself was created on a virtual thread? or what if there are no threads available in the common thread pool?
Well, one of the threads that a parallel stream uses to execute the stream pipeline is the thread that requests its execution. Thus, if the thread that requests execution of a stream pipeline is a virtual thread, then at least some part of the stream pipeline may be executed on a virtual thread.

The following is a slightly modified version of the parallel summation program that I showed earlier. This version requests stream execution from a **virtual thread**:

```
public class TestClass{
```

```
  static String tid(){  return ""+Thread.currentThread().threadId()+"
("+Thread.currentThread().isVirtual()+")"; }

  public static void main(String[] args) throws Exception {
      Thread t = Thread.startVirtualThread(() → {

System.out.println("Tid:"+tid());
Long sum = Stream.of(1, 2, 3, 4).parallel().reduce(Long.valueOf(0),
  (oldsum, newelement) →{
    Long newsum = Long.sum(oldsum, newelement.longValue());
    System.out.println("T "+tid()+" Accumulating - Old sum = "+oldsum+"
New element = "+newelement+" New sum = "+newsum);
    return newsum;
  },
  (sum1, sum2)→ {
    System.out.println("T "+tid()+" Combining - "+sum1+" "+sum2);
    return Long.sum(sum1, sum2);
  });
System.out.println("sum = "+sum);
        });

        t.join(); //necessary for virtual thread, remember?
    }
}
```

It produces the following output:

```
Tid:21(true)
T 26(false) Accumulating - Old sum = 0 New element = 4 New sum = 4
T 24(false) Accumulating - Old sum = 0 New element = 2 New sum = 2
```

```
T 25(false) Accumulating - Old sum = 0 New element = 1 New sum = 1
T 21(true) Accumulating - Old sum = 0 New element = 3 New sum = 3
T 25(false) Combining - 1 2
T 21(true) Combining - 3 4
T 25(false) Combining - 3 7
sum = 10
```

Observe that one of the stream fragments was executed on the same thread (ID 21) as the one that requested stream execution and that thread is a virtual thread. The rest of the fragments are executed on platform threads.

21.3.6 Creating primitive streams using ThreadLocalRandom ☝

You have already seen the usage of `Math.random()` method generate random numbers. This method is properly synchronized and can be used safely from multiple threads. However, due to strong synchronization, it performs poorly when invoked from multiple threads simultaneously. The `java.util.concurrent.ThreadLocalRandom` class is an alternative that provides a more efficient way of generating a limited or an unlimited stream of random numbers (`int` s, `long` s, and `double` s) from multiple threads. The exam expects you to know the general usage pattern of this class:

```
IntStream is1 = ThreadLocalRandom.current().ints(); //returns an
unlimited stream of random ints
IntStream is2 = ThreadLocalRandom.current().ints(10, 1000); //returns an
unlimited stream of random ints between 10 and 1000 (not including
1000)
IntStream is3 = ThreadLocalRandom.current().ints(100);
```

```
//returns a stream of 100 random ints
```

Similar methods exist for producing a stream of `long`s as well as `double`s. This approach avoids the resource contention associated with `Math.random()`, by letting you acquire an independent instance of a `ThreadLocalRandom` class for each thread through its `current()` method.

21.3.7 Quiz ☝

Q. Identify correct statements about the following code:

```
StringBuilder sb = new StringBuilder();
IntStream is = ThreadLocalRandom.current().ints(0, 10);
is.parallel().takeWhile(i→sb.length()<2001).forEach(s→sb.append(s));
System.out.println(sb.length());
```

Select **3** correct options.

A. `sb` will contain exactly 2000 numbers.
B. The length of `sb` is unpredictable.
C. `sb` will contain exactly 2000 numbers if the parallel operation is removed.
D. An exception may be thrown.

Correct answer is **B, C, D**.

This code illustrates the pitfalls associated with stateful lambda expressions in parallel stream operations. `i→sb.length()<2000` and `s→sb.append(s)` are stateful lambdas because `sb` refers to a mutable object whose state is not managed by the stream pipeline but its state is accessed from stream operations. Therefore, when the stream pipeline is parallelized, multiple threads employed by the stream pipeline to process the elements of the stream will try to access and mutate `sb` simultaneously.

Since `sb` is not a thread safe object, calling `sb.length()` and `sb.append(s)` from multiple threads simultaneously will produce unpredictable results.

Removing `parallel()` will ensure that only a single thread is employed to access `sb` and in that case, exactly 2000 numbers will be appended to it. Hence, **B** and **C** are correct. A runtime exception may also be thrown by the methods of `StringBuilder` when multiple threads try to mutate it simultaneously. Hence, **D** is also correct.

21.4 Exercise ☝

1. Create an infinite `IntStream` whose first element is 2 and next element is previous element + 3.
2. Find the sum of 10th to 15th element (both inclusive) of the above stream.
3. Create an `IntStream` of 100 random ints. Find maximum, minimum, and average of these values.
4. Find the sum of even numbers as well as odd numbers of the above stream using reduce as well as collect methods. Use any class with two int fields to store the sums.
5. From a stream of 100000 random integers, filter out the multiples of 5 and print the remaining elements in increasing order. Do the same using sequential as well as parallel streams and compare their performance.
6. Create a stream of 100 random strings of lengths 2 to 10. Find the number of words in the stream for each length. Print a map where key is the length and value is a list of unique words of that length.